TigerGraph GraphRAG Hackathon – Round 3

Generalizable Context Optimization Challenge

Challenge Overview

In the previous rounds, teams demonstrated strong results in reducing token consumption and improving GraphRAG efficiency on datasets they selected and optimized for.

Round 3 tests whether those methodologies can be generalized.

All teams will work with the same dataset, question set, evaluation criteria, and baseline requirements. Teams may apply the techniques developed in earlier rounds or introduce new approaches.

The goal is to determine whether teams can apply what they learned in previous rounds to a heterogeneous, real-world dataset.

Teams may use a single context-optimization methodology where it performs consistently or combine multiple methodologies based on the document type, information structure, question type, or retrieval pattern.

Successful solutions should demonstrate how the optimization strategy adapts across structured financial facts, long-form policies, governance documents, and other unstructured content, while showing that graph-based retrieval contributes meaningfully to token reduction, evidence quality, and answer accuracy.

Timelines & Prizes

Timelines for Round 3:

- Launch: July 17th 2026
- Target delivery date: July 26th 2026

Three winners (around 500 dollars):

- **1st place —** ₹25,000
- **2nd place —** ₹15,000
- **3rd place —** ₹10,000

Dataset

TigerGraph will provide a normalized dataset and a standardized set of evaluation questions.

The dataset is intentionally heterogeneous and includes:

- Financial information suited to entities, relationships, triples, numerical facts, and structured retrieval.
- Long-form unstructured documents, including policies, governance materials, strategic priorities, and innovation goals.
- Information distributed across multiple documents and document types.
- Questions requiring factual retrieval, cross-document reasoning, evidence aggregation, and multi-hop analysis.

This reflects real enterprise environments, where knowledge systems must handle structured facts, narrative documents, policies, numerical data, and relationships spanning multiple sources.

Solutions should adapt to different document structures and question types rather than relying on dataset-specific assumptions.

Problem Statement

GraphRAG can improve answer quality by retrieving evidence through vector similarity, entities, graph relationships, neighborhoods, and multi-hop connections.

This higher-recall retrieval can uncover evidence that traditional RAG may miss. However, it can also return duplicate, overlapping, or marginally relevant context.

As a result, the LLM may receive substantially more context than it needs, increasing token consumption, latency, and cost without a proportional improvement in answer quality.

The challenge is to build a context-optimization approach that transforms GraphRAG retrieval into a smaller, more precise, and higher-quality evidence set before generation.

The optimized GraphRAG pipeline should preserve the information and relationships needed to answer the question while removing redundant, repetitive, and low-value context.

This is not simply a deduplication problem. Similar passages may contain different facts, dates, entities, exceptions, relationships, or supporting evidence. Successful solutions must distinguish between repeated information and complementary evidence.

Required Pipelines

Each team must implement and compare the following three pipelines.

1. LLM-Only Baseline

The question is sent directly to the selected language model without access to the provided dataset.

2. Traditional RAG Baseline

Evidence is retrieved using and supplied to the language model.

3. Optimized TigerGraph GraphRAG

Evidence is retrieved and optimized using graph relationships, vector search, or a combination of both.

TigerGraph Savanna must be used as both the graph database and vector database for the GraphRAG pipeline.

The GraphRAG implementation must make meaningful use of graph structure. Storing disconnected document chunks in TigerGraph or reproducing a vector-only pipeline will not be considered sufficient.

Controlled Evaluation Conditions

The 3 pipelines must be evaluated under a fair, transparent and reproducible experimental design.

Fixed Generation and Evaluation Conditions

- The same evaluation drive
- The same question set
- The same generation LLM and exact model version

- Temperature 0 or another documented deterministic configuration, including a fixed seed where supported
- The same maximum output-token limit.
- The same core system instructions, answer format, and output constraints. Any pipeline-specific context or citation instructions must be minimal and disclosed.
- The same evaluation methodology, judge model, judging rubric, and scoring thresholds.

Retrieval and Context-Optimized Variables

Retrieval, indexing, chunking, graph traversal, reranking, filtering, compression, and context-selection strategies may differ because these are the experimental variables being evaluated.

- Where vector retrieval is used, Traditional RAG and GraphRAG must use the same embedding model so the comparison isolates the contribution of graph retrieval and context optimization.
- Graph-specific techniques may be applied only to GraphRAG and do not need to be reproduced in Traditional RAG where they are not applicable.
- Every material difference between the pipelines must be disclosed and justified.
- Traditional RAG must be implemented as a competent, good-faith baseline with sensible chunking, indexing, retrieval, and top-k settings. Teams may not intentionally over-retrieve or under-retrieve to distort the comparison.
- A pipeline may use a reranker, filter, or compression mechanism that another pipeline does not use, provided the difference is disclosed and its contribution is demonstrated through ablation analysis.

Tuning and Test-Set Isolation

- All pipelines must be tuned using the same development question set, which must be disjoint from the held-out evaluation set.
- No pipeline may be tuned using held-out questions, held-out results, or information derived from the test set.
- Teams should apply a comparable tuning time or compute budget across the retrieval-based pipelines rather than counting tuning iterations.
- Final pipeline configurations must be frozen before the held-out evaluation begins, and all tuning decisions must be documented.

Token Accounting

Actual token usage from the model or API must be reported whenever available. Estimates may be used only when actual usage is unavailable and must be clearly identified.

For every question and pipeline, teams must report:

- System and instruction tokens.

- Question tokens.
- Retrieved context tokens.
- Tokens consumed by any LLM-based classification, reranking, filtering, compression, summarization, or routing step.
- Generated output tokens.
- Total inference tokens across all model calls required to answer the question.

One-time ingestion, extraction, embedding, and indexing token costs must be reported separately from per-question inference costs. Token-efficiency comparisons will primarily use total inference tokens, not generation tokens alone.

Answer Evaluation and Judging

A strict pass/fail result requiring a complete, factually correct, and adequately supported answer.

A graded answer-quality score that distinguishes fully correct, mostly correct, partially correct, and incorrect answers.

BERTScore F1 (secondary semantic-similarity metric). To keep results comparable, all teams must use this exact config and report **F1**, computed against the **official reference answer** (never the retrieved context): `bert-score==0.3.13` · `model_type="roberta-large"` · `lang="en"` · `rescale_with_baseline=True` · `idf=False`. The `roberta-large` model and `rescale_with_baseline=True` are mandatory - without them the scores aren't comparable across teams.

A citation and evidence-quality score.

The LLM judge must be blind to the pipeline that produced each answer and should use a different model from the generator where practical. Partial, unsupported, hallucinated, unanswered, or malformed responses will not receive a strict pass but must remain in the reported results.

Reproducibility and Traceability

- Run the complete held-out evaluation three times using the same frozen pipeline configurations.
- Report the mean and variation for nondeterministic metrics and identify metrics that are fully deterministic.
- Retain the raw per-question outputs from every run rather than reporting only the best run.
- Make every claim traceable to committed source code, pinned dependency versions, configuration files, exact prompts, retrieved evidence, token-usage records, and raw evaluation outputs.

Context-Optimization Approaches

Teams are free to design their own methodology. Potential approaches include:

- Semantic deduplication.
- Diversity-aware ranking, such as Maximum Marginal Relevance.
- Graph-aware relevance scoring.
- Entity- and relationship-based evidence selection.
- Evidence compression or summarization.
- Adaptive chunking and hierarchical chunk selection.
- Retrieval-chunk versus generation-chunk optimization.
- Question classification and adaptive retrieval.
- Graph-based evidence fusion.
- LLM-assisted evidence filtering.
- Deterministic or learned reranking.
- Novel graph-based or hybrid optimization methods.

There is no prescribed architecture. Teams are encouraged to challenge existing assumptions and demonstrate new approaches.

Bonus

- Quantify the token-efficiency and answer-quality impact of each major optimization mechanism through ablation analysis
- Identify which document types and question types benefit from each mechanism
- Report interactions between mechanisms when their combined effect differs from their individual effects

---

Benchmark Validity Requirements

Token reduction will only receive credit when the benchmark and comparison are valid.

The following requirements must be satisfied:

- The official question set will primarily require evidence from the provided dataset
- Any questions on which the LLM-only baseline performs as well as or better than a retrieval-based pipeline must remain in the results and be analyzed; they do not automatically invalidate an otherwise valid benchmark.
- RAG and GraphRAG should demonstrate that access to the dataset improves answer quality over the LLM-only baseline.
- Token savings must not be achieved through a material reduction in accuracy.
- The GraphRAG pipeline must demonstrate that graph-derived information contributes to retrieval, evidence selection, reasoning, or context optimization.

- All provided questions must be evaluated.
- Teams may not remove unfavorable questions or failed results.
- Failed, unanswered, and malformed responses must remain in the reported results.
- Citations must refer to evidence actually supplied to the model.

Results that fail these requirements may be excluded from ranking regardless of the reported token savings.

Evaluation and Scoring

The winner will not necessarily be the team reporting the largest token reduction or the highest standalone accuracy score. The winning submission will most convincingly demonstrate that graph-based retrieval reduces total inference-token consumption while preserving or improving answer quality, with every claim traceable to committed code and raw results and reproduced across independent runs.

A modest, rigorously demonstrated saving will be valued more highly than a large, unverifiable one. Judges will weigh the rigor of the controlled comparison, reproducibility, transparency of total inference-token accounting, separately reported ingestion and indexing costs, and the quality-per-token trade-off—not the headline reduction figure alone.

1. Answer Quality and Accuracy — 40%

Teams will be evaluated on:

- BERTScore F1 against the provided reference answers.
- Strict pass rate and graded LLM-as-judge results
- Correct handling of factual, numerical, policy, and multi-document questions.
- Accuracy relative to both the LLM-only and traditional RAG baselines.

2. Token Efficiency — 30%

Teams will be evaluated on:

- Reduction in retrieved context tokens and total inference tokens compared with Traditional RAG
- Total inference-token consumption across all model calls and the complete question set
- Separate disclosure of one-time ingestion, extraction, embedding and indexing costs
- Consistency of token reduction across question and document types

- Efficiency gains achieved without sacrificing accuracy
- Token reduction may be discounted or receive no credit when accompanied by a material decline in answer quality.

3. Evidence and Citation Quality — 15%

Teams will be evaluated on:

- Citation correctness.
- Citation completeness.
- Whether the cited evidence supports the answer.
- Preservation of complementary evidence from multiple sources.
- Reduction of duplicate and repetitive context.

4. Graph Contribution and Experimental Rigor — 10%

Teams will be evaluated on:

- Meaningful use of TigerGraph relationships and graph traversal.
- Evidence that graph-derived signals contribute to the results.
- Fair and controlled comparison of the three pipelines.
- Reproducible per-question results.
- Ablation analysis showing the impact of major optimization components.

5. Generalizability and Solution Quality — 5%

Teams will be evaluated on:

- Performance across the heterogeneous dataset.
- Adaptability to different document and question types.
- Architecture quality and maintainability.
- Clarity of the implementation and documentation.

Required Results

For every question and pipeline, teams must record:

- Generated answer.
- Retrieved document and chunk identifiers.
- Evidence supplied to the LLM.
- Citations included in the answer
- Token count:
    - System and instruction token counts

    - Tokens used by any LLM-based routing, classification, reranking, filtering, compression or summarization step
    - Output token count
    - Total inference tokens across all model calls
- BERTScore F1.
- Strict pass/fail result and graded LLM-as-judge score
- Citation and evidence-quality score
- Latency, where available
- Errors or failed responses

Teams must also provide aggregate results across the complete question set.

One-time ingestion, extraction, embedding and indexing costs must be reported separately from per-question inference costs

An ablation analysis is required to demonstrate the contribution of the graph and the major context-optimization components. For example, teams may compare results with graph-based scoring disabled, context optimization disabled, or individual optimization techniques removed.

Deliverables

1. TigerGraph Savanna Instance

A working Savanna instance containing:

- The Round 3 dataset
- The graph schema and relationships
- Vector indexes used by the GraphRAG pipeline
- Queries and graph algorithms required to run the solution
- Access for judges to review and test the implementation

2. GitHub Repository

A complete repository containing:

- Source code for all three pipelines.
- Data ingestion and transformation code.
- Retrieval and context-optimization logic.
- Evaluation scripts.
- Installation and execution instructions.
- Configuration examples that do not expose credentials or API keys.

3. Three-Pipeline Application

A working application that allows judges to run and compare:

- LLM-only
- Traditional RAG
- Optimized TigerGraph GraphRAG

4. Evaluation Dashboard

A dashboard comparing the three pipelines across the complete question set, including:

- Token usage
- Token reduction relative to traditional RAG
- BERTScore F1
- Strict pass rate and graded LLM-as-judge results
- Citation and evidence quality
- Latency, where available
- Failed or unanswered questions
- Aggregate and per-question results

The dashboard should make it easy to determine whether token reduction was achieved without sacrificing answer quality.

5. Technical Architecture and Technology Stack

Documentation describing:

- Language model used
- Embedding model used
- Vector database used by each applicable pipeline
- TigerGraph graph schema
- Entity- and relationship-extraction methodology
- Chunking and indexing strategies
- Retrieval, reranking, filtering, and context-optimization techniques
- Evaluation tools and frameworks
- Agentic frameworks, orchestration tools, and other libraries used

6. Methodology Presentation

A presentation covering:

- Architecture of the three pipelines
- Context-optimization methodology
- Use of graph structure and graph-derived signals

- Experimental controls
- Token-efficiency, accuracy, and citation results
- Ablation findings
- Challenges and limitations
- Lessons learned
- Generalization to the heterogeneous Round 3 dataset

Teams will present their methodology, application, and results to the judging panel.

7. Blog or Social Media Post

A public blog or social media post describing:

- The Round 3 challenge
- The team's approach
- How TigerGraph was used
- Key results and lessons learned

The post should acknowledge participation in the TigerGraph GraphRAG Hackathon and may link to the team's public repository, demonstration, or technical write-up.

Resources Provided by TigerGraph

TigerGraph will provide:

- The normalized Round 3 dataset
- The official evaluation questions
- Reference or ground-truth answers
- Gemini API keys
- TigerGraph Savanna credits
- Evaluation guidance and required result formats

Teams may use additional open-source libraries and frameworks, provided that TigerGraph Savanna is used for graph storage, graph retrieval, and vector search in the GraphRAG pipeline.

Participant Teams

- Riseon
- MasterGraph

- MasterMind
- Aayush & Anushka
- Nextin
- Jeswant
- Code of Duty
- Sri Lakshami
- Think Respire
- HopLogic

TigerGraph will meet with the teams before Round 3 to review the challenge, clarify the evaluation requirements, and discuss the architectures and token-reduction approaches developed during the previous rounds.